INTRODUCTION TO PYTHON

Django Web Framework

Building dynamic websites with Python



Dr. Prab M
Department of Computer Science and Engineering
Amrita Vishwa Vidyapeetham, Chennai Campus

OVERVIEW

What You Will Learn



Web Basics

How browsers and servers talk using HTTP



Meet Django

What a framework is and why we use Django



MVT Architecture

Model, View, Template and the request flow



Core Building Blocks

URLs, views and templates



Models and Database

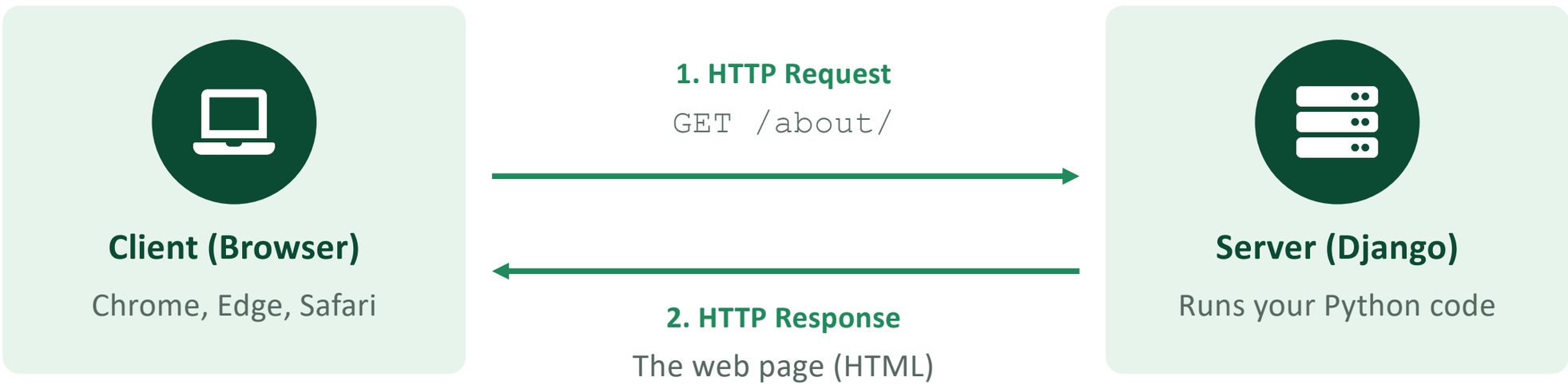
ORM, migrations and the admin site



Forms and Practice

Handling user input and hands-on labs

How the Web Works



1. You type a web address
2. The server runs Django code
3. Your browser shows the page

HTTP Request and Response

Request: browser to server

- Method: GET (read) or POST (send data)
- URL: the address, such as /about/
- Headers: browser details, cookies
- Body: form data (POST only)

Response: server to browser

- Status code: shows the result
- Headers: content type, cookies
- Body: the HTML page, an image or data

Common status codes

200 OK | **302 Redirect** | **404 Not Found** | **500 Server Error**

Static vs Dynamic Websites



Static website

- Fixed HTML files
- Same page for every visitor
- Changing content means editing files

Example: a simple college brochure page



Dynamic website

- Pages are built on demand
- Content comes from a database
- Can change for each user and request

Example: a results portal or an online shop

Django is used to build dynamic websites.

What is a Web Framework?

A framework is a ready-made toolkit of code that solves the common problems of building websites.



Think of building a house

You do not make the bricks and doors yourself. You use ready-made parts and design the rooms.

A framework gives you



URL routing



Database access



HTML templates



Forms



Security



User login

What Is Django?

2005

Released as open source

Python

Written entirely in Python

MVT

Model, View, Template design

- Free and open source
- Named after jazz guitarist Django Reinhardt
- Built for fast development of secure sites
- Used by Instagram and Mozilla

Companies Built on Django

- **Instagram** (The world's largest Django deployment)
- **Zapier** (Automation & workflow platform)
- **The Washington Post** (The original birthplace of Django)
- **Robinhood** (Fintech & stock trading platform)
- **Bitbucket** (Source code hosting & collaboration tool)
- **Eventbrite** (Global ticketing & event marketplace)
- **Disqus** (Global web comment hosting service)
- **Nextdoor** (Neighborhood social network)
- **NASA** (Uses Django-based systems for content management)
- **The UK National Health Service (NHS)** (Powers public health platforms)
- **Google** (Uses Django for official blog systems)
- **Mozilla** (Powers add-on and developer platforms)

Why Use Django?

Batteries included

Admin, login, forms and ORM come built in

Fast development

Build working sites in hours, not weeks

Secure by default

Guards against CSRF, XSS and SQL injection

Scalable

Works for small projects and very large sites

Clean Python code

Readable, reusable and easy to maintain

Strong community

Great documentation and many add-on packages

MVT: Django's Architecture



Model

Handles the data

- Describes the data
- Talks to the database
- Lives in models.py



View

Handles the logic

- Receives the request
- Prepares the response
- Lives in views.py



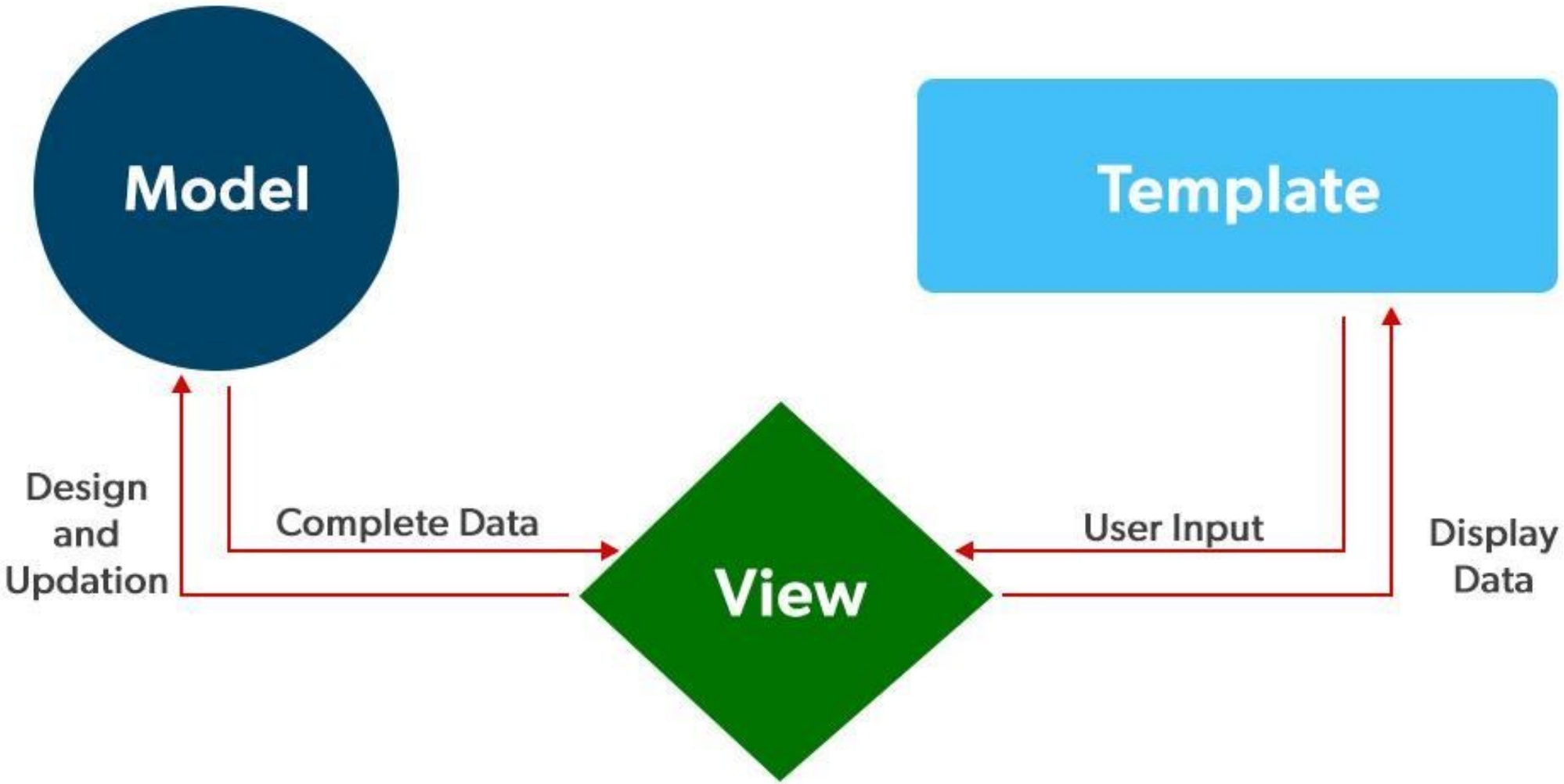
Template

Handles the display

- Controls the page look
- HTML with Django tags
- Lives in templates/

Django term	Model	View	Template
Classic MVC name	Model	Controller	View

The URL dispatcher (urls.py) sends each request to the right view.


Model
Template
Design
and
Updation
Complete Data
View
User Input
Display
Data

MVT Workflow

- A user sends a request through the browser.
- Django matches the request URL using urls.py.
- A view processes the request.
- The view interacts with models to retrieve or update data.
- The view passes data to a template.
- The template generates HTML.
- Django returns the rendered response to the browser.

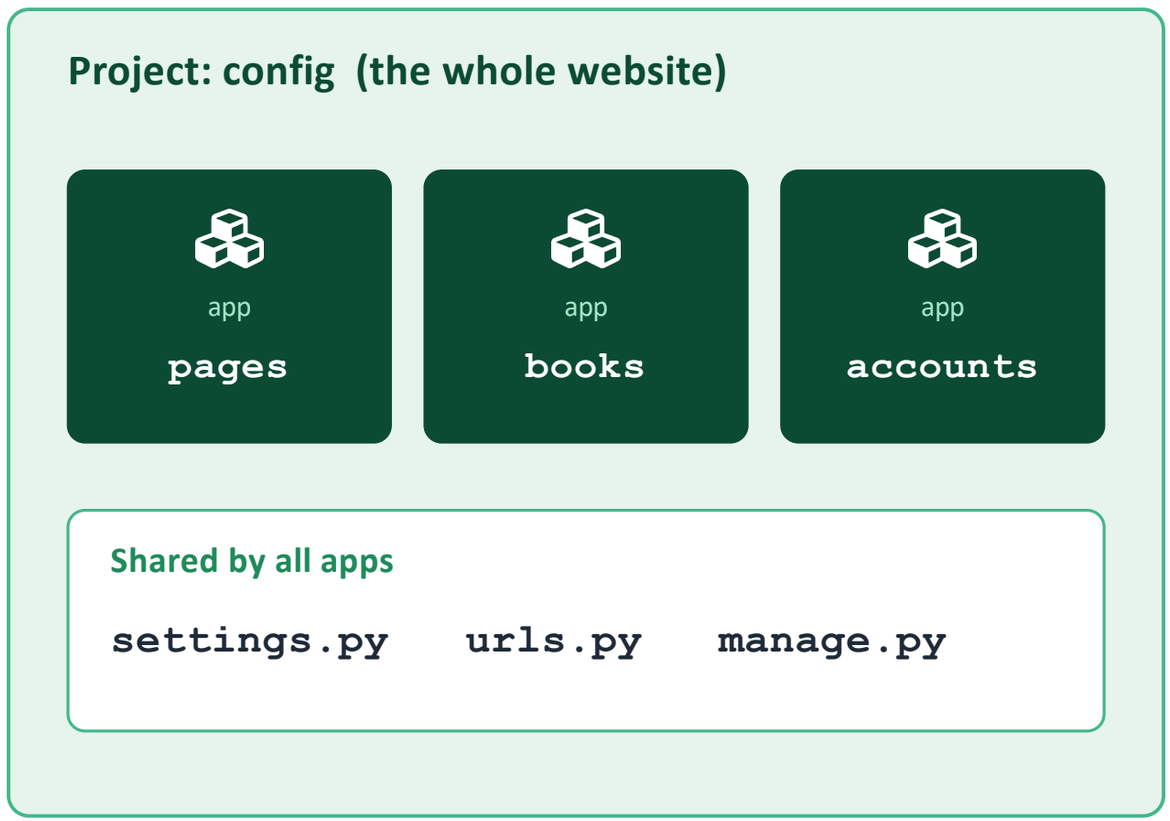
The Life of a Request

1. **Browser** — Sends a request for /books/
2. **urls.py** — Finds the matching URL pattern
3. **views.py** — Runs the view function
4. **models.py** — Reads data from the database
5. **Template** — Builds the HTML page
6. **Browser** — Shows the finished page

EXAMPLE

You open /books/. Django matches the URL and runs the book_list view. The view asks the Book model for all books, the model reads the database, the template turns the data into HTML, and your browser shows the table.

Project vs App



Project

The whole website and its settings

App

One feature, such as pages or books

Many apps

One project can hold many apps, and apps can be reused

Anatomy of a Django Project

```
practice_project/
├── manage.py
├── db.sqlite3
├── config/
│   ├── settings.py
│   └── urls.py
└── books/
    ├── models.py
    ├── views.py
    ├── urls.py
    ├── admin.py
    ├── forms.py
    ├── migrations/
    ├── templates/books/
    └── static/books/
```

File or folder	Purpose
`manage.py`	Runs commands for the project
`settings.py`	Project configuration
`urls.py`	URL patterns (main and per app)
`models.py`	Database tables as Python classes
`views.py`	Functions that handle requests
`admin.py`	Admin site registration
`templates/`	HTML pages
`static/`	CSS, images and JavaScript

Setting Up Django

1. `python3 -m venv venv` — Create a private environment for this project
2. `source venv/bin/activate` — Switch it on (Windows: venv\Scripts\activate)
3. `pip install django` — Install Django inside the environment
4. `django-admin startproject config .` — Create the project (the dot means "here")
5. `python manage.py startapp books` — Create an app for one feature
6. `python manage.py runserver` — Start the test server at 127.0.0.1:8000

Mac commands are shown. On Windows, use python instead of python3.

URLs: Routing Requests

pages/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('about/', views.about, name='about'),
]
```

1. **The address pattern**

   An empty pattern is the home page; about/ is /about/

2. **The view to run**

   A function from views.py

3. **A nickname**

   Used later in links: {% url 'home' %}

Django checks the patterns from top to bottom and uses the first match.

Views: The Logic

books/views.py

```
from django.shortcuts import render
from .models import Book

def book_list(request):
    books = Book.objects.all()
    return render(
        request,
        'books/book_list.html',
        {'books': books},
    )
```

A view is a Python function that:

- Receives the request
- Gets data from models
- Returns a response

`HttpResponse`	sends plain text
`render`	builds a page from a template
`redirect`	sends the visitor elsewhere

Templates: The HTML

books/templates/books/book_list.html

```
<h1>My Book Library</h1>
<p>Total: {{ books|length }}</p>

{% for book in books %}
  <p>{{ book.title }}</p>
{% empty %}
  <p>No books yet.</p>
{% endfor %}

<a href="{% url 'add_book' %}">Add</a>
```

Syntax	Meaning
`{{ value }}`	Displays a value
`{% tag %}`	Runs logic, such as a loop
`{% for x in y %}`	Repeats HTML for each item
`{% url 'name' %}`	Builds a link from a URL name
`{% load static %}`	Turns on static file tags
`{{ x|length }}`	A filter changes how a value shows

Models and the ORM

books/models.py

```
class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.CharField(max_length=100)
    year = models.IntegerField()
```



Database table: books_book

id	title	author	year
1	Wings of Fire	Kalam	1999
2	The Alchemist	Coelho	1988
3	Malgudi Days	Narayan	1943

ORM: query the database using Python

`Book.objects.all()`

Get every row

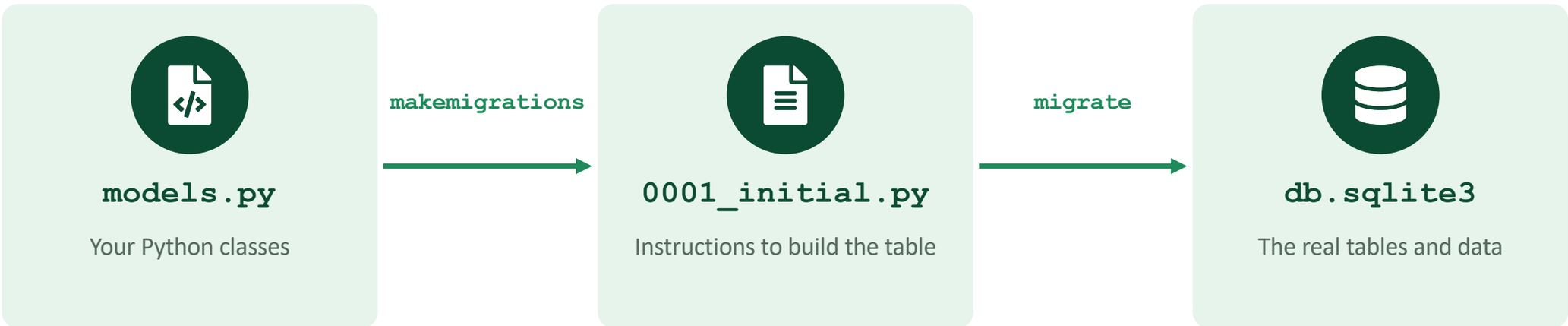
`Book.objects.filter(year=1999)`

Get rows that match

`Book.objects.get(id=1)`

Get one row by its id

Migrations: Model to Database



Analogy

makemigrations is the architect's plan. migrate is the construction work.

Rule to remember

Changed models.py? Run makemigrations and migrate again.

The Django Admin Site

1 Create an admin account

```
python manage.py createsuperuser
```

2 Register the model in admin.py

```
books/admin.py

from django.contrib import admin
from .models import Book

admin.site.register(Book)
```

3 Open /admin/ and log in

```
http://127.0.0.1:8000/admin/
```

You get for free

- Add, edit and delete records
- Search and filter data
- User login and permissions
- No HTML to write

Forms: Taking User Input

1. **GET request**

   The view shows an empty form

2. **User clicks Save**

   The browser sends a POST request

3. **`form.is_valid()`**

   Django checks the submitted data

4. **`form.save()`**

   Data is stored, then the user is redirected

books/forms.py

```
class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['title', 'author']
```



CSRF token

Add {% csrf_token %} inside every POST form. It blocks fake submissions.

Settings and Static Files

settings.py controls the project

Setting	Purpose
INSTALLED_APPS	Apps the project uses
DATABASES	Which database to use
TEMPLATES	Where templates are found
STATIC_URL	Web address of static files
DEBUG	Shows detailed errors
SECRET_KEY	Secret code for security

Static files never change



CSS Images JavaScript

template

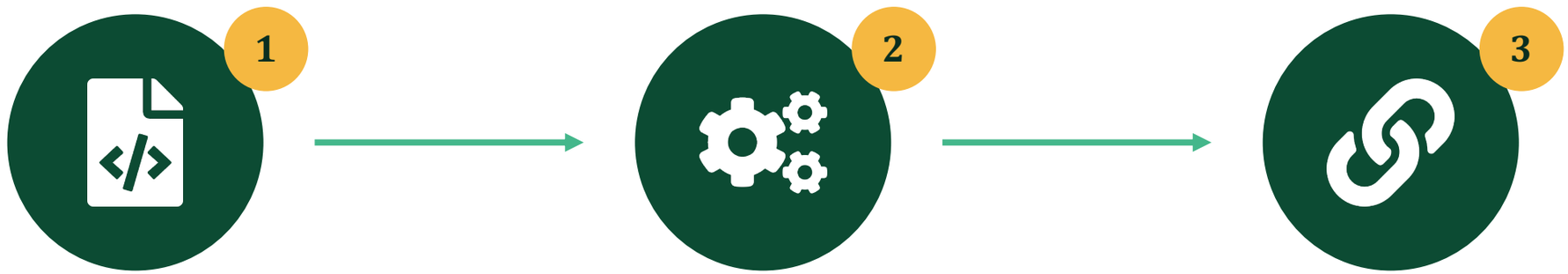
```
{% load static %}
<link rel="stylesheet"
  href="{% static 'pages/style.css' %}">
```

Keep DEBUG = True only while developing.

manage.py Commands

Command	What it does	When to use it
`python manage.py startapp name`	Creates a new app	Adding a feature
`python manage.py runserver`	Starts the test server	Viewing in the browser
`python manage.py makemigrations`	Prepares database changes	After editing models.py
`python manage.py migrate`	Applies those changes	After makemigrations
`python manage.py createsuperuser`	Creates an admin login	Once per project
`python manage.py shell`	Opens Python with your project loaded	Trying ORM queries
`python manage.py test`	Runs automated tests	Checking your code

Adding a Page: The 3-Step Pattern



Template

Write the HTML in the templates folder

View

Write a function that renders the template

URL

Add a path that points to the view

Every page you build in Django follows this pattern.

Hands-On Practice



Practice 1: Static Website

A two-page personal site with CSS

You learn: URL to View to Template

Time: About 90 minutes

URL | View | Template | Static files



Practice 2: Database Website

My Book Library with a database

You learn: Model to Admin to Form

Time: About 2 hours

Model | Migration | Admin | Form

Follow the practice manuals step by step on your own laptop.

Reading Errors Like a Developer



1. Read the last line of the error first
2. Check that every file is saved
3. Check the spelling of names and paths
4. Is (venv) showing in the terminal?

Error message	Usual cause
`TemplateDoesNotExist`	Wrong template folder path
`NoReverseMatch`	URL name does not match
`no such table`	Forgot to run migrate
`ImportError`	File unsaved or name misspelt
`CSRF verification failed`	Missing csrf_token

Key Takeaways

1

Django is a Python framework for building dynamic websites

2

MVT: Model is data, View is logic, Template is HTML

3

Every page needs a template, a view and a URL

4

Migrations turn models into database tables

5

The admin site and forms save enormous effort

6

Real learning happens by practising on your laptop

REVIEW

Quick Check

1. What is the difference between a project and an app?
2. What do M, V and T stand for in Django?
3. Which file connects a web address to a view?
4. What is the difference between makemigrations and migrate?
5. Why do we use a virtual environment?
6. Which three things are needed to add a new page?

Resources to keep learning

- Official documentation: docs.djangoproject.com
- Official tutorial: docs.djangoproject.com/en/stable/intro/tutorial01/
- Django Practice Manuals 1 and 2 (shared in class)

